

The `lutorch_ex` library

lookup-table layers: tables, projections, cartridges, and results

(draft)

2026-10-05 (DRAFT — refine next session)

1 The basic idea: lookup tables over an LSH

A lookup table (LUT), here, is a learnable map built from a *discrete* address: hash the input to a small integer with a locality-sensitive hash (LSH), and read a stored vector at that address. The hash is cheap — a handful of sign bits — so a “read” is a gather plus a few adds, not a matrix multiply. `lutorch_ex` builds many such tables and sums them into an expressive learnable map. This is a general construction; using it as a drop-in feed-forward layer (the setting benchmarked later) is just one application of it, not its defining purpose.

One table. Work in a space $z \in \mathbb{R}^{d_{\text{in}}}$. A table picks `nap` *anchors*, each defining one scalar form u_j of z ; the *signs* of the u_j form an `nap`-bit address $c \in [0, K)$ into $K = 2^{\text{nap}}$ cells, and each cell holds a learnable vector $W[c] \in \mathbb{R}^{d_{\text{out}}}$. Each sign bit is one LSH bit — which side of a hyperplane $u_j = 0$ the input lies on; inputs on the same side of all `nap` hyperplanes share a cell. The margins u_j (how far each form is from flipping its sign) measure the address’s confidence — a small margin means a bit is near its boundary.

Primitive hyperplanes only. The hyperplanes a table can use are deliberately *primitive*: each passes through the origin (zero bias) and has trivial coefficients. Two modes:

- **pairs** (`anchor_mode="pairs"`): $u_j = z[a_j] - z[b_j]$ — coefficients in $\{-1, +1\}$ on two coordinates (the *anchor pairs*).
- **single** (`anchor_mode="single"`): $u_j = z[a_j]$ — a single $+1$ coefficient (the even simpler *single anchors*), so the bit is just the sign of one coordinate.

Both share the same read; only how the `nap` forms are built changes. These primitive forms are all a table sees: *arbitrary* hyperplanes ($w^\top z$ with a general w and bias) do not exist at this level. The next section recovers them, with an input projection.

Many tables, many heads. The layer stacks `tph` tables per head over G head-groups ($G = \text{n_groups} = \max(h_{\text{in}}, h_{\text{out}})$). A head’s read *sums* the addressed cell(s) over its `tph` tables (optionally weighted — see the cartridges), giving a per-group output $[B, G, d_{\text{out}}]$, which a routing step maps to $[B, h_{\text{out}}, d_{\text{out}}]$ (identity when $h_{\text{out}} = G$; a group-sum when $h_{\text{out}} = 1$).

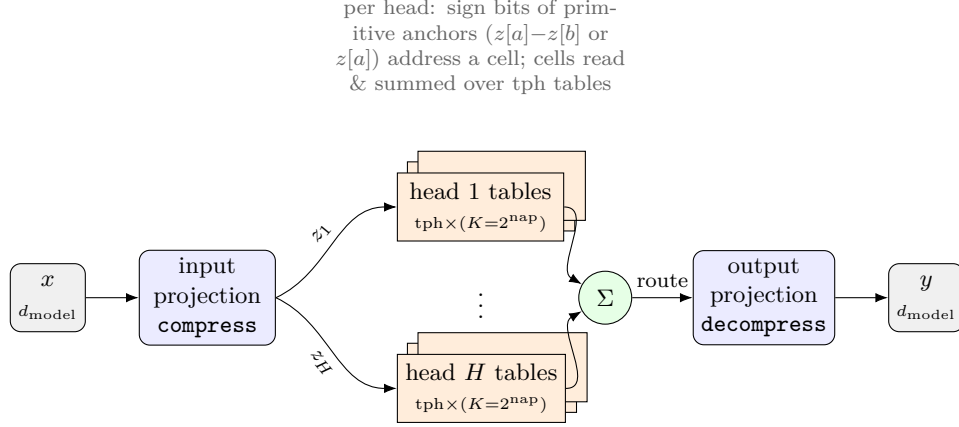


Figure 1: The ProjectionLUT data path. The input x is projected by **compress** into H per-head spaces; each head’s primitive-anchor LSH addresses its own stack of **tph** discrete tables ($K = 2^{\text{nap}}$ cells each); the per-head reads are summed and routed, then **decompress** projects back to y .

Geometry and the base. The immutable geometry is LUTSpec: $\mathbf{h_in}, \mathbf{h_out}, \mathbf{tph}, \mathbf{nap}, \mathbf{d_in}, \mathbf{d_out}$, with $\mathbf{n_cells} = 2^{\text{nap}}$, $\mathbf{in_features} = \mathbf{h_in} \mathbf{d_in}$, $\mathbf{out_features} = \mathbf{h_out} \mathbf{d_out}$. The abstract base **ManifestoLUT** owns the learnable cell tables $W \in \mathbb{R}^{G \times \text{tph} \times K \times d_{\text{out}}}$ and the addressing (**_addresses**, which uses only spec/anchors/powers — never the weight *values*) and the routing invariant (**_route**). The faithful default init draws cell tables per head from a CPU generator seeded (seed + $h + 1$), Uniform $[-\sigma, \sigma]$ with $\sigma = \mathbf{weight_init_std}$ (realised std $\sigma/\sqrt{3} \approx 5.8\text{e-}4$ at the 10^{-3} default), reproducing the OLD LightMHL draw bit-for-bit.

2 ProjectionLUT: input and output projections

On its own a table reads a narrow per-head vector. **ProjectionMHL** (the “ProjectionLUT” wrapper, drawn in Figure 1) turns a cartridge into a plain $d_{\text{model}} \rightarrow d_{\text{model}}$ FFN with a linear map on each side, for two reasons:

- **input projection (compress)** — a Linear $d_{\text{model}} \rightarrow \mathbf{in_features}$. It *shapes the hashing*: the primitive anchors now act on **compress**(x) rather than on raw model dimensions, so a primitive form $z[a] - z[b]$ or $z[a]$ on the projected vector is an *arbitrary* learned hyperplane $w^\top x$ (with bias) on the original x . This is exactly how the primitive-only tables get general hyperplanes.
- **output projection (decompress)** — a Linear $\mathbf{out_features} \rightarrow d_{\text{model}}$ lets the tables live in a narrow d_{out} space and be *compressed*: the cells are small, and the projection expands the summed read back to model width.

The forward is $z = \text{compress}(x)$ (reshaped to $[B, h_{\text{in}}, d_{\text{in}}]$); $y = \text{cartridge}(z)$; $\text{out} = \text{decompress}(y)$. Either side may be switched off (Identity) when the widths already match.

Init and first-backward warmup. **compress** $\sim \mathcal{N}(0, 0.02)$; **decompress** is *zeroed*. Since $\partial \text{out} / \partial y = W_{\text{dec}} = 0$ at init, the first backward sends no main-loss gradient to the cartridge or **compress**; only **decompress** (and the tables, through the cell-smoothness penalty of §5.1) move on step 0, and the input side “warms up” once **decompress** leaves zero — reproducing OLD’s zero-contribution FFN at init. Both inits are guarded so a **meta-device** build allocates nothing.

The decompress-bake capability. One hook matters later: a cartridge may expose `decompress_scale()` (a length-`out_features` vector), advertised by the `SupportsDecompressBake` protocol. `ProjectionMHL` folds it into the `decompress` matrix (scaling column k = scaling cartridge output channel k , at no extra cost). This is the route by which a quantised cartridge’s per-channel scale can be absorbed into the dense projection — used by deployment (§4.3).

3 Cartridges: different math, one contract

Everything above is fixed: the geometry, the addressing, the per-head sum, the routing, the projection wrapper. A *cartridge* is the swappable lookup *math* inside that contract — how a table turns its addressed cell(s) and margins into an output. All cartridges share the read skeleton (each group addresses one cell c_g per table from the sign pattern, the read sums over the `tph` tables, then `_routes` $[B, G, d_{\text{out}}] \rightarrow [B, h_{\text{out}}, d_{\text{out}}]$); they differ in whether the read is hard or blended, whether it is weighted by a learned confidence score, and whether it is quantised.

Implementation: native ops. Every cartridge runs as pure, portable torch by default. The *fused* cartridges additionally offer a tier-2 CUDA fast path, reached through one choke point, `_native_ops.native_manager()`, which resolves (1) the co-located JIT extension `lutorch_ex_lprojection` (`lprojection` / anchor-pairs kernels, built from `cartridges/csrc/` via `cpp_extension.load`), else (2) the pure/tier-1 torch path. The int8 read has its own JIT extension, `lutorch_ex_pow2_int8_read`. “Native path” below means this tier-2 fast path; it never changes the numerical result, only the speed, and falls back to pure torch when CUDA/nvcc/ninja are absent.

4 The cartridges

4.1 ConfidenceLUT

Gen-3 scored read. A learned-margin confidence score weights each table’s cell contribution. With margins u and $m = |u|$, the per-table score is

$$s = \left(\sum_j m_j \right) \exp\left(\gamma \sum_j \log \sigma(\beta m_j) \right),$$

$\beta = e^{\log \beta}$, $\gamma = e^{\log \gamma}$ learnable (per layer; g frozen at 0 in the champion). A large score means every margin is large, i.e. the address is confident. The $n=2$ read blends the addressed cell c_1 and the runner-up c_2 (flip the least-certain bit, the smallest margin $u^* = \min_j u_j$) with a softmax over the deciding margin: $v = \sigma(2u^*/\tau)$, contributing $s[vW[c_1] + (1-v)W[c_2]]$; $n=1$ reads only the scored cell $sW[c_1]$. Gradients flow to W , β , γ (and τ for $n=2$).

```
from spiky.lutorch_ex import LUTSpec, ConfidenceLUT, ProjectionMHL
spec = LUTSpec(h_in=8, h_out=8, tph=64, nap=8, d_in=48, d_out=48)
cart = ConfidenceLUT(spec, seed=1, read_top_n=2, beta_init=2.0, gamma_init=1.0,
                    read_tau_init=0.5, read_tau_learnable=True)
ffn = ProjectionMHL(cart, d_model=384) # drop-in d_model -> d_model FFN
```

4.2 QuantisedConfidenceLUT (extended)

The `p2_int8` quant-aware twin of `ConfidenceLUT`. Same confidence score/blend, but the read is quantisation-aware with a straight-through estimator (STE):

- **Cell tables** are fake-quantised to int8 with a per-(group,channel) *power-of-two* scale: for group g , channel k , an integer exponent $e_{g,k}$ (`head_chan_exponents`) gives $\hat{W} = \text{round}(W/2^e) \cdot 2^e$, clamped to int8; STE passes the identity gradient to the fp32 master W .
- **Blend weights** are rounded to powers of two: $2^{k'}$ and $2^{k'-q}$ with $k' = \text{round}(\log_2 s - c_q)$ and $q = \text{round}(2|u^*|/(\tau \ln 2))$ (stop-grad exponents; the STE carries the exact $(s \cdot v, s \cdot (1-v))/s$ gradient).
- **Integer read** (`forward_int` / the deployed path): int8 cell rows, int32 shift-add in fixed point (shift = 6, `FIXED_POINT_SHIFT`), accumulator in units 2^{-6} , then scaled by 2^{e-6} per (group,channel). Equals the fake-quant forward value to round-off.

Exact constants (from `sources/quantisation_simple_norowops.md`). `quant_cfg`: $b = 8$ bits, offset $o = 0$. Blend: $q = \text{clamp}(\lfloor 2u^*/(\tau \ln 2) + \frac{1}{2} \rfloor, 0, J)$ with $J = 64$, $c_q = \log_2(1+2^{-q})$ for $q < C = 8$ (else 0); the second cell is dropped when $q > Q = 3$. The weight exponent $k' = \text{round}(\log_2 s - c_q)$ lives in an $L = 8$ window $[-3, 4]$ ($k_{\max} = 4$, 3 bits): $k' < -3$ skips the table (weight 0, gradient kept), $k' > 4$ clamps (fires on $\sim 0.001\text{--}0.007\%$ of tables). The per-(head,channel) weight exponent is $e_{h,c} = \lceil \log_2 \max_{t \in h, \text{cells}} |W_t[\cdot]_c| \rceil - (b-1) + o$ — the max over the *head*'s tables, so one exponent is shared across them (int8 $o = 0$, never saturates; int4 $b = 4, o = -1$). `FIXED_POINT_SHIFT` = 6 because $k' + 6 \geq 0$ and $k' + 6 - q \geq 0$ for all $k' \geq -3, q \leq 3$, so the integer read is pure left-shifts-and-adds. Measured cost: $\sim +0.003$ bpb vs the unquantised soft read; footprint 151.0 MB fp32 $\rightarrow$ 37.75 MB int8 ($4\times$) $\rightarrow$ 18.87 MB int4 ($8\times$).

What the cartridge emits for deployment. `to_deployment()` returns the int8 packed table plus the per-out-feature power-of-two scale 2^{e-6} ; it requires `read_top_n=2` and $h_{\text{out}} = G$ (fan-in would need a pre-route scale). The next subsection turns that into a compact checkpoint.

```
from spiky.lutorch_ex import QuantisedConfidenceLUT, ProjectionMHL
cart = QuantisedConfidenceLUT(spec, seed=1, quant_mode="p2_int8", read_top_n=2,
                             beta_init=2.0, gamma_init=1.0, read_tau_init=0.5)
ffn = ProjectionMHL(cart, d_model=384)
```

4.3 Deployment: the .lxq format

An inference-only, cartridge-agnostic path packages a trained model into a compact checkpoint. `export_deployment(model, "m.lxq")` walks the model and, for each `ProjectionMHL` whose cartridge emits a payload (above), folds the per-out-feature power-of-two scale 2^{e-6} *exactly* into a copy of that FFN's `decompress` weight — using the `decompress-bake` hook of §2; a power-of-two only touches the fp32 exponent field, so the fold is lossless. This works because one exponent is shared across a head's tables (it factors out of the table sum) *and* nothing sits between the table sum and `decompress`. The deployed cartridge then returns the *raw* int32 accumulator (its own `decompress_scale()` is `None`: the scale now lives, statically, in the stored weight). The .lxq file is a safetensors container. `load_deployment(path, make_skeleton, device)` builds the architecture on the `meta` device (no fp32 allocation) and rebuilds each compacted FFN from a stored format tag via the leaf `deploy_registry` (tag $\rightarrow$ `from_deployment`), so the fp32 LUT table is never allocated; the quantised footprint drops $\sim 4\times$ (int8).

```
from spiky.lutorch_ex import export_deployment, load_deployment
export_deployment(model, "m.lxq") # int8 bake + fold pow2 scale into decompress
dep = load_deployment("m.lxq", make_skeleton, device="cuda").eval() # fp32 table never
    allocated
```

4.4 FusedManifestoHardLUT

Hard value read $\sum_t W[c_t]$ (the argmax cell per table, no score weighting), fused. The native path uses the `lprojection` non-smooth backward; tier-1 fuses the per-table gather + sum with `embedding_bag`. [TBD — exact forward/backward from `manifesto_hard.py` + `_native_ops.NativeHard`.]

```
from spiky.ltorch_ex import FusedManifestoHardLUT, ProjectionMHL
ffn = ProjectionMHL(FusedManifestoHardLUT(spec, seed=1), d_model=384)
```

4.5 FusedManifestoSoftLUT

Soft temperature-blended read; the native path uses `lprojection_forward_smooth` and its native smooth backward. [TBD — soft-blend math from `manifesto_soft.py`.]

4.6 FusedSoftSignHardLUT / FusedSoftSignSmoothLUT

Gen-2 soft-sign cartridges. The two-cell blend weight is $w = \sigma(-2\rho/T_{\text{sel}})$ with $\rho = |m|/(T_{\text{soft}} + |m|)$ (deciding margin m , learned temperatures $T_{\text{soft}}, T_{\text{sel}}$); the hard variant uses the straight-through value, the smooth one the true blend. The surrogate’s m/T gradients are a one-launch `softsign_surrogate_grad` kernel (tier-1: eager). [TBD — confirm against `softsign*.py` + `_native_softsign.py`; `lut_mechanisms` has the Gen-2 forms.]

```
from spiky.ltorch_ex import FusedSoftSignHardLUT, FusedSoftSignSmoothLUT, ProjectionMHL
ffn = ProjectionMHL(FusedSoftSignSmoothLUT(spec, seed=1), d_model=384)
```

5 Experiments

Model. Each block of a small GPT (6 blocks, model width $d_{\text{model}} = 384$, $\approx 68.2\text{M}$ params) has its feed-forward layer replaced by a `ProjectionLUT` wrapping the cartridge under test (the LUT islands run in fp32). All seven runs share the champion topology ($h=8$, `tph`= 64, `nap`= 8, $d_{\text{in}} = d_{\text{out}} = 48$), seed 1, and the faithful small-std init.

5.1 Regularisation and training recipe

Two regularisers are part of the champion recipe (applied on `ltorch_ex`, not the skeleton).

Cell total-variation ($\mathcal{R}_{\text{TV}}$, $\lambda = 10$). View each table’s $K = 2^{\text{nap}}$ cells as the vertices of an nap -dimensional hypercube ($[G\text{-tph}, \underbrace{2, \dots, 2}_{\text{nap}}, d_{\text{out}}]$). The penalty sums the squared difference (channels

summed in) over every Hamming-1 cell pair — a size-2 `diff` along each of the `nap` bit-axes, each pair counted once — normalised by the pair count:

$$\mathcal{R}_{\text{TV}} = \frac{1}{n_{\text{tables}} \text{nap} 2^{\text{nap}-1}} \sum_{(c,c') \text{ Hamming-1}} \|W_c - W_{c'}\|_2^2.$$

Added to the loss as $\lambda \mathcal{R}_{\text{TV}}$ ($\lambda = 10$) through a separate backward; differentiable w.r.t. W only; adds nothing to the forward. Hamming-1 neighbours are addressed by inputs differing in a single anchor-sign bit, so penalising their squared difference smooths the table — a one-bit address flip changes the read little (a locality prior over the discrete lookup). (Bit-for-bit the reference `LightMultiHeadLUT.cell_tv`.)

Head (table) dropout ($r = 0.2$). Inverted dropout at whole-table granularity: one independent Bernoulli per (sample, group=head, table) on $[B, G, \text{tph}]$, keep prob $1 - r$, survivors rescaled $1/(1 - r)$; dropped tables contribute 0. Train-time + grad-only (eval reads all tables); the same keep-mask multiplies the backward. It discourages co-adaptation among a head’s `tph` tables.

Hyperparameters. Cell-TV $\lambda = 10$, head-dropout 0.2; AdamW lr 3×10^{-4} , weight decay 0.1 (none on the LUT tables), 48,000 steps, cosine schedule (10% warmup, $0.1 \times \text{floor}$), sequence length 512, device batch 12, total batch 24,576 tokens, eval every 500. wandb project `Spiky`, group `lutorch_ex`; per-run artifacts (config, metrics, summary) are under `experiments/lutorch_ex/`.

5.2 The seven-config sweep

All seven runs are complete. Final `val_bpb` (best→worst):

Configuration	cartridge / mode	wandb run	final val_bpb
ConfidenceLUT $n=2$	scored, float	<code>lutorch_ex_abl47_conf_1004_1148</code>	1.10606
QuantConfidence $n=2$	scored, p2_int8	<code>lutorch_ex_abl47_quant_1004_1148</code>	1.10724
ConfidenceLUT $n=1$	scored, float, $n=1$	<code>lutorch_ex_abl47_conf_n1_vm_1004_1806</code>	1.11737
FusedSoftSignSmooth	soft-sign, smooth	<code>lutorch_ex_abl47_fss_smooth_vm_1004_1806</code>	1.12941
FusedManifestoSoft	soft	<code>lutorch_ex_abl47_fms_wk_1004_1510</code>	1.13949
FusedManifestoHard	hard	<code>lutorch_ex_abl47_fmh_wk_1004_1510</code>	1.15152
FusedSoftSignHard	soft-sign, hard	<code>lutorch_ex_abl47_fss_hard_wk_1004_1510</code>	1.15647

Reading the results. The scored `ConfidenceLUT` line wins: $n=2$ is best at **1.10606**, and int8 power-of-two quantisation of it costs only +0.0012 bpb (1.10724) — consistent with the $\sim +0.003$ bpb upper bound of §4.2 — for a $4\times$ smaller table. Dropping the blend to a single cell ($n=1$) costs +0.0113. The non-scored families trail: the soft reads (FusedSoftSignSmooth 1.12941, FusedManifestoSoft 1.13949) beat their hard counterparts (FusedManifestoHard 1.15152, FusedSoftSignHard 1.15647). So a soft/blended read and a learned confidence score each help, and they stack.

Learning curves. [TBD — insert the `val_bpb` learning-curve figure (7 curves); data is in `experiments/lutorch_ex/*/metrics.csv`.]